

CYPRESS UI AUTOMATION & WORKFLOW REPORT

SauceDemo Web Application Automation Project
Assignment No. 6

Submitted By	Farooque Sajjad
Department	Quality Assurance
Organization	10Pearls
Assignment	Assignment 6 - Cypress Automation Framework
Test Tool	Cypress Automation Framework
Submission Date	28-05-2026

1. Introduction

The objective of this assignment was to automate the SauceDemo web application using the Cypress automation framework. The project focused on implementing maintainable and scalable UI automation using industry-standard practices such as the Page Object Model (POM), reusable custom commands, and structured test organization.

The SauceDemo application is commonly used for automation practice and provides an excellent platform for validating login functionality and product navigation workflows.

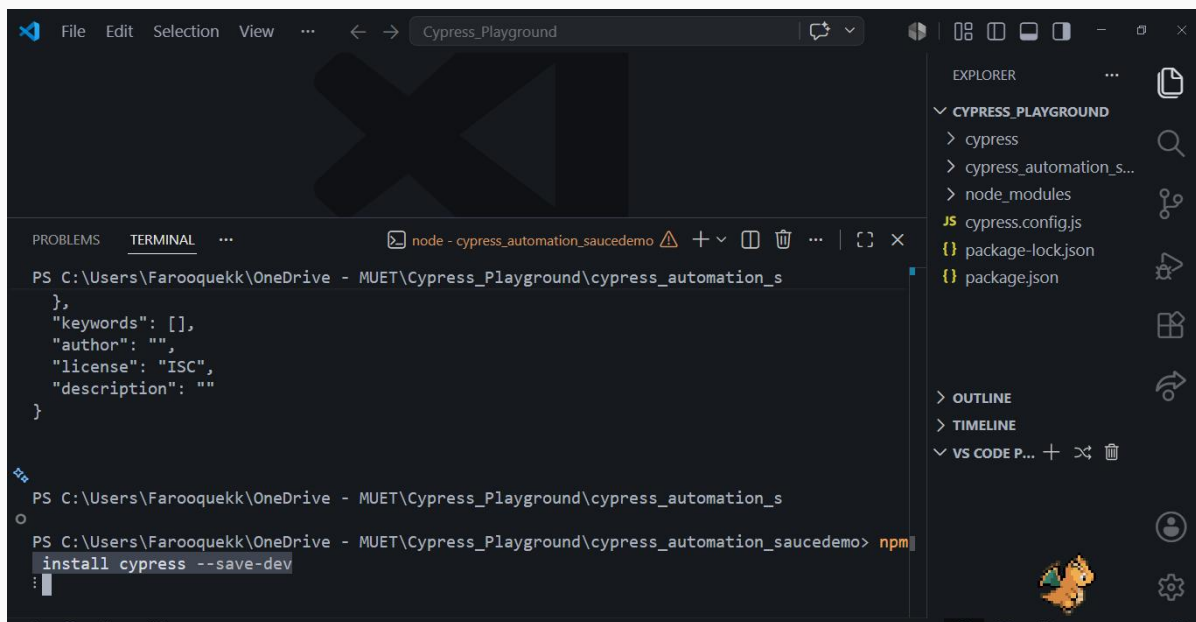
2. Project Setup

The project was initialized using Node.js environment dependencies and native Cypress libraries packages to support a local manual and headless engineering architecture.

Steps Performed:

1. Created dedicated project workplace directory.
2. Initialized a fresh Node.js project environment via npm init.
3. Installed local development Cypress dependency cleanly.
4. Generated default workspace scaffold Cypress framework structure.
5. Configured global Cypress JSON/JS environmental configuration values.

Cypress Installation Proof:

A screenshot of the Visual Studio Code (VS Code) interface. The main editor window shows a file named 'package.json' with the following content:

```
{,
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}
```

 The terminal window at the bottom shows the command prompt with the following commands:

```
PS C:\Users\Farooquekk\OneDrive - MUET\Cypress_Playground\cypress_automation_s
PS C:\Users\Farooquekk\OneDrive - MUET\Cypress_Playground\cypress_automation_s
PS C:\Users\Farooquekk\OneDrive - MUET\Cypress_Playground\cypress_automation_saucedemo> npm
install cypress --save-dev
```

 The Explorer panel on the right shows the project structure:

- CYPRESS_PLAYGROUND
 - cypress
 - cypress_automation_s...
 - node_modules
 - cypress.config.js
 - package-lock.json
 - package.json
- OUTLINE
- TIMELINE
- VS CODE P...

Filename: 01_Installing_Cypress.png

3. Framework Architecture

The internal engineering directory skeleton maps structural separation cleanly to isolate config, utilities, objects, data schemas, and specifications perfectly.

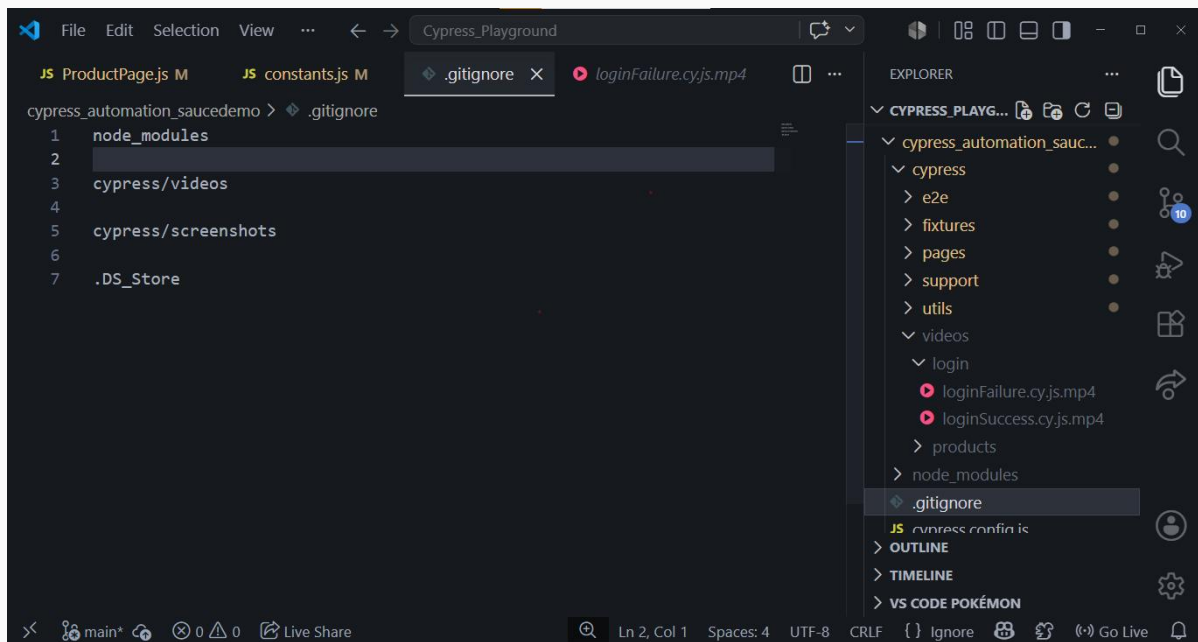
Folder Tree Schema:

cypress/e2e/	Contains execution test specs files covering sequential web flows (*.cy.js)
cypress/fixtures/	Maintains dynamic static JSON values and test context credentials datasets cleanly
cypress/pages/	Holds Object element identifiers mapping individual functional screen elements
cypress/support/	Hosts dynamic globally registered user operations custom scripts and commands configurations
cypress/utils/	Encapsulates common modular engineering algorithmic helpers / asset generators
cypress/videos/	Automatically captures and stores full-length browser screen recordings (.mp4) of test suite runs for validation and triage workflows.
node_modules/	Maintains system standard system specific background dependencies libraries cleanly

Architectural Benefits Realized:

- **High Maintainability:** Separation of element locators prevents script fragmentation when interfaces modify.
- **Improved Readability:** The testing scripts mirror behavioral step flows abstracting technical selector properties.
- **Better Scalability:** Additional specification runs easily inherit baseline custom properties seamlessly.
- **Reduced Code Duplication:** Modular functions share baseline elements avoiding script redundancy.

Framework Directory Architecture Screenshot:



Filename: 02_project_structure.png

4. Page Object Model Implementation

Page Object Model (POM) architecture patterns were introduced to strictly isolate physical HTML elements and action handlers directly away from raw testing assertions code files.

Pages Designed and Core Structural Responsibilities:

1. LoginPage Class

- Maintains clean text field targets for primary user text inputs.
- Exposes action helpers tracking multi-role text insertions (Username/Password text inputs).
- Tracks dynamic application system click behaviors mapping the master Form Submission trigger.
- Captures custom error elements tracking error notifications banner validations cleanly.

2. InventoryPage Class

- Validates master layout visibility following active verification context benchmarks.
- Handles multi-item catalog access parsing product listing layouts details structurally.
- Triggers seamless product detail view state transitions accurately.

3. ProductPage Class

- Ensures atomic verification metrics extract title, item data, description values easily.
- Validates exact item price matching benchmarks directly inside text layers properties.

5. Custom Commands Implementation

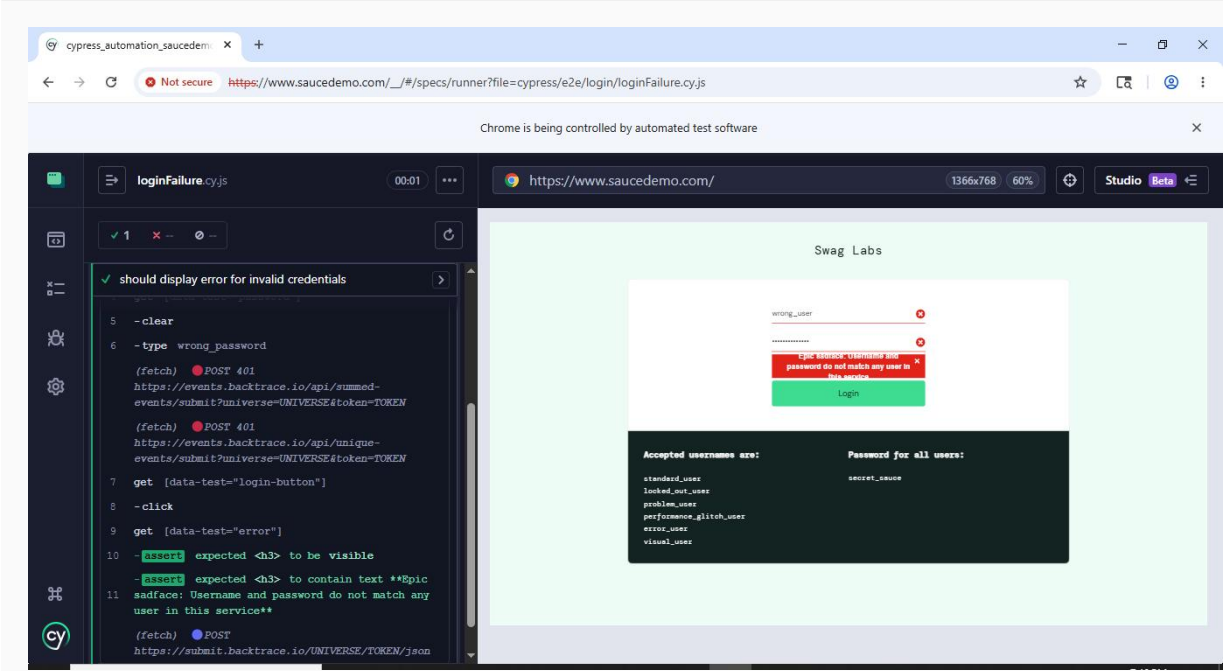
Global utility functions extensions were customized within the support directory stack using Cypress system middleware commands endpoints to compress multi-step repetitive browser sequences down into clean reusable chain handles.

- **cy.login(username, password):** Performs automated browser navigation and form entry using parameter inputs cleanly across raw variable permutations contexts.
- **cy.loginAsStandardUser():** Bypasses structural preparation stages by instantly injecting standard authorized baseline user credentials to optimize child workflow runs.

6. Test Cases Implemented

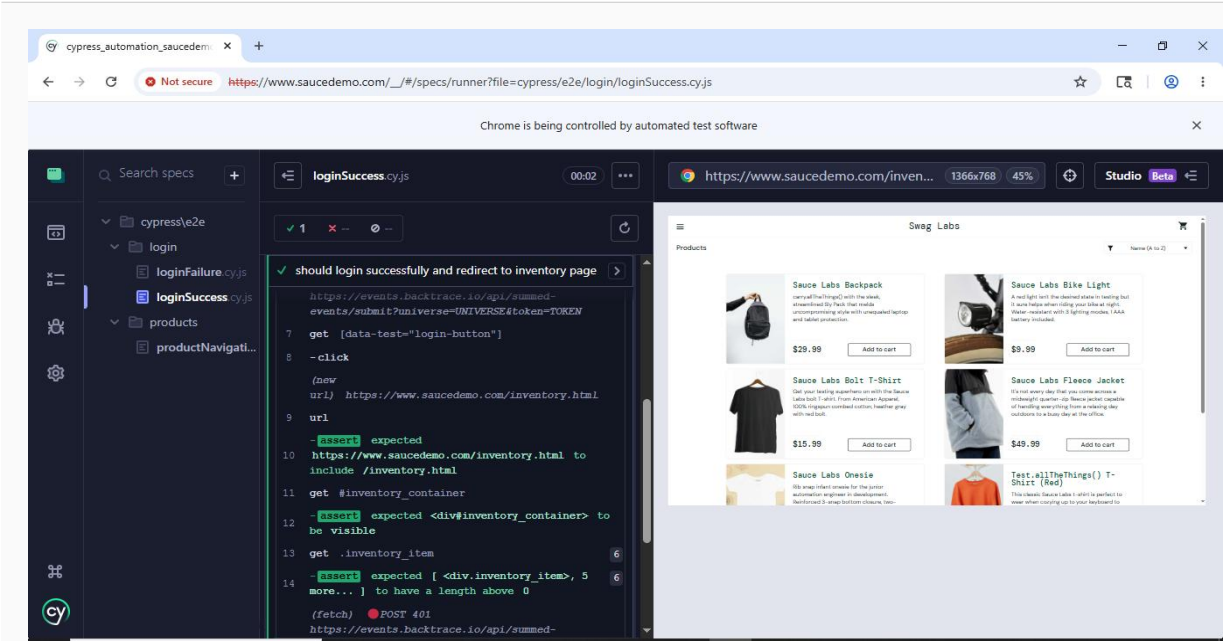
Test ID	Scenario Objective	Test Verification Type	Expected System Behaviour
TC-CY-001	Validate login system prevents unauthorized user sessions utilizing incorrect credentials format parameters.	Negative Validation	Displays clean invalid string error message banners contextually.
TC-CY-002	Validate credential verification paths authorize valid profile profiles input vectors cleanly.	Smoke / Functional	Redirects the application viewport layout cleanly onto Inventory boards contextually.
TC-CY-003	Validate index view selections cascade details perfectly towards detailed overview screens.	End-To-End Functional	Loads dedicated matching detail content configurations safely.

Test Case 1: Login Failure Validation Proof



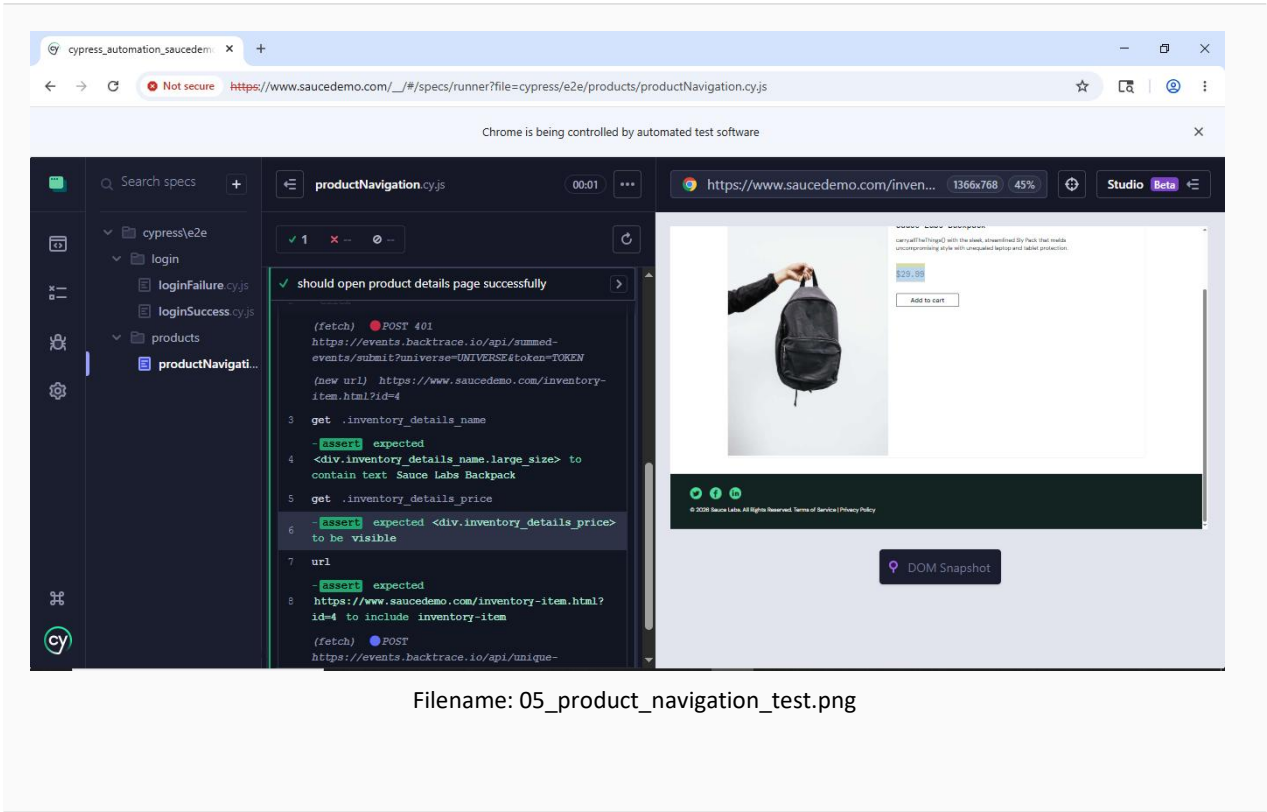
Filename: 03_login_failure_test.png

Test Case 2: Login Success Validation Proof



Filename: 04_login_success_test.png

Test Case 3: Product Navigation Validation Proof



7. Test Execution Results

The unified automated framework validation run triggered flawlessly against the local application systems environment staging target using explicit terminal run parameters execution mode script arguments.

Framework Invocation Terminal Script:

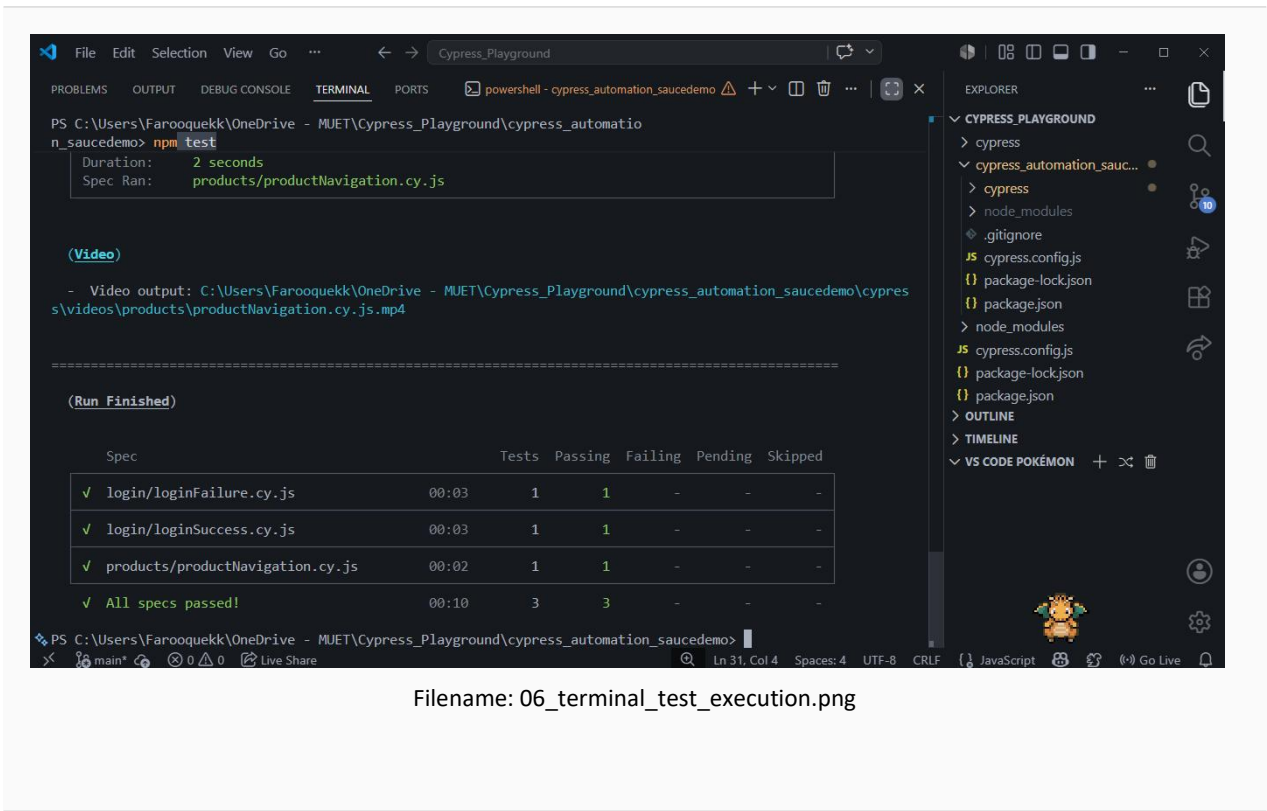
npm test (command)

Execution Matrix Summary Results Table:

Module Sequence	Automated Scenario Objective	Execution Status	Pass Percentage
TC-CY-001	Login Failure Validation Scenarios Run	PASSED	100%

TC-CY-002	Login Success Validation Scenarios Run	PASSED	100%
TC-CY-003	Product Navigation Validation Scenarios Run	PASSED	100%

Terminal Output Console Run Execution Proof Stills:



Filename: 06_terminal_test_execution.png

8. Conclusion

This assignment successfully demonstrated Cypress UI automation using modern automation engineering practices. The framework was implemented using Page Object Model architecture, reusable custom commands, and organized test structures. All required scenarios including login validation, successful authentication, and product navigation were automated and executed successfully.

The resulting framework is scalable, maintainable, and aligned with industry-standard automation development practices.